

LABORATORY

LAB 05

Fundamentals of Analog Write

Controlling LED Brightness with PWM Using EduFramework

S32K144 · MaaZEDU

EduFramework

FPT University

Contents

1	Introduction	2
1.1	Lab Overview	2
1.2	Objectives	2
2	Background	2
2.1	Pulse Width Modulation	2
2.2	Period and Frequency	2
2.3	Duty Cycle	3
2.4	PWM and LED Brightness	3
3	Hardware Setup	3
3.1	Hardware Used	3
3.2	LED Mapping Used in the Lab Exercise	4
4	EduFramework API	4
4.1	<code>analogWrite()</code>	4
5	Lab Exercise	5
5.1	Requirements	5
5.2	Program	5
5.3	Verification	6
6	Extension	6
6.1	RGB Color Mixing with PWM	6
6.2	Smooth Color Transition Hints	7
7	References	7

1 Introduction

1.1 Lab Overview

Digital Output in the previous labs used two states, `HIGH` and `LOW`, to turn an output on or off. However, many applications require intermediate levels of control, such as changing LED brightness or controlling motor speed instead of only turning the output fully on or off.

Pulse Width Modulation (PWM) is a technique for controlling the average power delivered to a load by rapidly switching a signal between the on and off states. Instead of directly changing the voltage amplitude of the digital signal, PWM changes the proportion of time the signal remains in the on state during each period [1]. This lab focuses on the relationship between PWM period, duty cycle, and LED brightness.

1.2 Objectives

OBJECTIVES

The objectives of this laboratory are:

1. Explain the basic principle of Pulse Width Modulation (PWM).
2. Describe the concepts of period, frequency, and duty cycle of a PWM signal.
3. Explain the relationship between duty cycle and the observed brightness of an LED.
4. Use `analogWrite()` to control a PWM level.
5. Build and verify an application that continuously fades an LED in and out.

2 Background

2.1 Pulse Width Modulation

PWM generates a digital signal that periodically switches between two states: on and off. PWM is described as a method of controlling the average power delivered to a load by rapidly switching the signal on and off; a PWM signal is commonly represented as a square wave with a defined period [1].

An important characteristic of PWM is that the logic amplitude does not need to change. Instead, the amount of time the signal remains in the on state during each period is varied. Therefore, PWM is still a digital signal over time, not a continuous analog voltage level.

2.2 Period and Frequency

A PWM signal repeats according to a fixed period. Let the time during which the signal is in the on state be T_{ON} and the time during which it is in the off state be T_{OFF} . The period T is defined as:

$$T = T_{\text{ON}} + T_{\text{OFF}}$$

The frequency f indicates the number of periods repeated in one second and is related to the period by:

$$f = \frac{1}{T}$$

Period and frequency describe how fast the signal repeats, while duty cycle describes the proportion of on time within each period. These are three basic quantities used to describe a PWM signal [1].

2.3 Duty Cycle

Duty cycle is the ratio between the time the signal remains in the on state and the total duration of one period. This value is usually expressed as a percentage:

$$D = \left(\frac{T_{ON}}{T} \right) \times 100 \%$$

For example, a duty cycle of 25% means that the signal is in the on state for one quarter of the period; a duty cycle of 50% means that the on and off times are equal. As duty cycle increases, the on time in each period increases, and the average power delivered to the load also increases [1].

Duty cycle	On time within the period	Meaning
0%	None	The signal always remains in the off state.
25%	One quarter of the period	The on time is shorter than the off time.
50%	One half of the period	The on and off times are equal.
75%	Three quarters of the period	The on time is longer than the off time.
100%	The entire period	The signal always remains in the on state.

Table 5.2.1. Relationship between duty cycle and the on time of a PWM signal

2.4 PWM and LED Brightness

PWM is commonly used to control LED brightness. When the switching frequency is sufficiently high, consecutive on and off events are not observed as separate states; instead, the LED is perceived at a brightness level that depends on the proportion of on time [1].

Therefore, for the same PWM period, changing the duty cycle makes it possible to change the observed brightness without switching among multiple logic-voltage levels. This is the principle used in the lab exercise to fade the LED in and out.

The relationship between duty cycle and the actual brightness of an LED is not necessarily perfectly linear. For a real LED, duty cycle does not correspond perfectly linearly to the output brightness; in simple applications, an approximate relationship can be used for control [4].

Note

PWM controls the proportion of time a digital signal remains in the on state. `analogWrite()` in this lab is used to control PWM; the API name does not mean that the pin generates a continuous analog voltage level.

3 Hardware Setup

3.1 Hardware Used

This lab directly uses the onboard LED on the MaaZEDU Development Board and does not require any additional components or external circuits.

Hardware	Description
MaaZEDU Development Board	Development board using the S32K144 microcontroller.
USB Cable	Connects the board to the computer for power and program uploading.

Table 5.3.1. Hardware used in the lab exercise

3.2 LED Mapping Used in the Lab Exercise

EduFramework provides the Logical Pin `LED_RED` to access the onboard red LED. This lab uses this LED as a PWM output; no external LED connection is required.

Component	Logical Pin	MCU Pin	Function
Onboard red LED	<code>LED_RED</code>	<code>PTD15</code>	PWM Output

Table 5.3.2. LED mapping used in the lab exercise

Before controlling the brightness, `LED_RED` must be configured as `OUTPUT` using `pinMode()`.

4 EduFramework API

This lab introduces one new API, `analogWrite()`, which allows the program to control a PWM level using a Logical Pin and a numeric value instead of directly working with low-level PWM configuration [3].

4.1 `analogWrite()`

`analogWrite()` writes a PWM value to a Logical Pin that supports PWM. In EduFramework, the input value is in the range from `0` to `255`; this value represents the control level from off to maximum brightness for the LED used in the lab exercise.

`analogWrite`

Writes a PWM value to the specified Logical Pin.

Syntax

```
analogWrite(pin, value);
```

Parameters

Parameter	Type / accepted value	Description
<code>pin</code>	PWM-capable Logical Pin	The pin to be controlled using PWM, for example <code>LED_RED</code> .
<code>value</code>	<code>0</code> to <code>255</code>	The PWM level to set. <code>0</code> corresponds to the LED being off, while <code>255</code> corresponds to maximum brightness in the context of this lab.

Return value

No return value.

Example:

```
analogWrite(LED_RED, 128U);
```

Intermediate values allow the duty cycle to be adjusted to different levels. When the value is gradually changed over time, the LED brightness also changes gradually in the corresponding direction.

5 Lab Exercise

5.1 Requirements

Build a program that makes `LED_RED` gradually increase from the off state to maximum brightness, then gradually decrease back to the off state and repeat continuously.

The program must satisfy the following requirements:

- Configure `LED_RED` as `OUTPUT`.
- Increase the PWM value sequentially from `0` to `255` so that the LED fades in.
- Decrease the PWM value from `255` to `0` so that the LED fades out.
- Use a `5 ms` delay between consecutive levels so that the effect can be clearly observed on the hardware.

5.2 Program

In `src/main.c`, implement the program that has been verified on the hardware as follows:

```
#include "Arduino.h"

int main(void)
{
    int pwmValue = 0U;

    setup();

    pinMode(LED_RED, OUTPUT);

    while (1)
    {
        /* Fade in: 0 -> 255 */
        for (pwmValue = 0U; pwmValue <= 255U; pwmValue++)
        {
            analogWrite(LED_RED, pwmValue);
            delay(5U);
        }

        /* Fade out: 255 -> 0 */
        for (pwmValue = 255U; pwmValue > 0U; pwmValue--)
        {
            analogWrite(LED_RED, pwmValue);
            delay(5U);
        }

        analogWrite(LED_RED, 0U);
    }

    return 0;
}
```

Listing 5.5.1. Program for fading an LED in and out using PWM

`setup()` initializes the fundamental components of EduFramework. Then, `pinMode(LED_RED, OUTPUT)` configures the red LED as an output using the knowledge introduced in Lab 01.

The first `for` loop increases `pwmValue` from `0` to `255`. Each value is passed to `analogWrite()`, after which the program waits `5 ms` before moving to the next level. As a result, the PWM level increases in small steps and the LED gradually becomes brighter.

The second `for` loop performs the reverse process: `pwmValue` decreases from `255` to nearly `0`, causing the LED to gradually become dimmer. The `analogWrite(LED_RED, 0U)` statement after the loop ensures that the LED returns to the off state before the next cycle begins.

The application flow can be summarized as:

`0` → gradually increase PWM value → `255` → gradually decrease PWM value → `0` → repeat.

5.3 Verification

Build and upload the program to the MaaZEDU Development Board. Observe `LED_RED` over several consecutive cycles.

EXPECTED RESULT

The red LED starts in the off state, gradually increases to maximum brightness, and then gradually decreases back to the off state. This process repeats continuously, and the change between consecutive brightness levels can be observed as a smooth brightness transition.

The value of `delay(5U)` can be temporarily changed to observe how the time between PWM-value updates affects the speed of the effect. This only changes the update rate of the values in the program and does not change the definition of the PWM duty cycle.

6 Extension

MaaZEDU provides three LED channels—red, green, and blue—through the Logical Pins `LED_RED`, `LED_GREEN`, and `LED_BLUE`. These three components can be controlled independently to extend brightness control to RGB color mixing.

6.1 RGB Color Mixing with PWM

RGB color mixing is based on three color components: red, green, and blue. The output color of an RGB LED is produced by changing the levels of the R, G, and B components; the components are combined according to the principle of additive color mixing, and the level of each component can be controlled using PWM duty cycle [4].

At the application level, a color can be represented as a set of three PWM values (`R`, `G`, `B`). Several basic combinations are shown below:

Resulting color	Red	Green	Blue
Red	255	0	0
Green	0	255	0
Blue	0	0	255
Yellow	255	255	0
Cyan	0	255	255
Magenta	255	0	255
White	255	255	255

Table 5.6.1. Example RGB values using a control range from 0 to 255

The value combinations above represent ideal combinations at the control level. The observed color may differ in practice because the characteristics and brightness of each LED are not completely identical. Different LED colors have different electrical characteristics and current-to-brightness relationships, so systems that require high color accuracy must consider the specific hardware characteristics [4].

6.2 Smooth Color Transition Hints

If the program completely turns off the current color before turning on the next one, the change appears as an abrupt transition. To create a smooth color transition, the PWM values of two or three channels can be changed simultaneously in many small steps.

For example, to transition from red to green:

- Gradually decrease the value of `LED_RED` from 255 to 0.
- At the same time, gradually increase the value of `LED_GREEN` from 0 to 255.
- Keep `LED_BLUE` at 0 throughout the transition.

The same idea can be applied sequentially to create the following color sequence:

Red → Yellow → Green → Cyan → Blue → Magenta → Red.

Note

The extension does not require a color science algorithm or color calibration. The objective is to apply the same PWM principle from the main exercise to multiple channels and gradually change the RGB values to create color mixing and smooth transitions.

Extension: Write a program using `LED_RED`, `LED_GREEN`, and `LED_BLUE` to create a continuous color-transition sequence. Do not switch directly from one color to another; instead, gradually change the PWM values of the relevant channels so that the color transition can be observed continuously.

7 References

- [1] University of Wisconsin-Madison, *Pulse Width Modulation*, ECE353 - Introduction to Microprocessor Systems.
- [2] NXP Semiconductors, *S32K1xx Series Cookbook*, AN5413, Rev. 5, 2020.
- [3] Arduino, *analogWrite()*, Arduino Language Reference.
- [4] Texas Instruments, *MSP430 Software RGB LED Control Design Guide*, TIDU761, 2015.